Blockchain Technologies 2

FINAL PROJECT

Full-Stack Decentralized Protocol — Team Capstone

1. PROJECT OVERVIEW

The Final Project is the capstone of the Blockchain Technologies 2 course. Teams must design, implement, test, audit, deploy, and demonstrate a complete production-grade decentralized protocol that integrates ALL technical domains covered across Lectures 1–10.

This is not a tutorial re-implementation. Teams are expected to behave as a real engineering team: plan work, split it, run code review, ship in increments, and produce a product another engineer could audit, extend, and deploy to mainnet without rewriting it.

1.1 Team Structure

— Team size: 2–3 students. Solo submissions are NOT accepted.
— Each team member must have a clearly documented area of ownership.
— Team composition is locked at the end of Week 6. No transfers afterward.

1.2 Project Scope

Each team chooses ONE of the following scenarios. The chosen scenario must be approved by the instructor by the end of Week 6. Changing scenario mid-project is not permitted. Regardless of the chosen scenario, the team MUST cover every mandatory technical component listed in Section 3.

— **Option A — DeFi Super-App:** an AMM + lending protocol + tokenized yield vault (ERC-4626), priced via Chainlink price feeds, governed by a DAO, indexed via The Graph, deployed on an L2.
— **Option B — GameFi Economy:** an ERC-1155 in-game item economy with crafting, a marketplace AMM for fungible resources, an NFT rental vault, Chainlink VRF for loot drops, DAO-governed game parameters (drop rates, crafting costs), L2 deployment.
— **Option C — RWA Tokenization Platform:** ERC-20 asset-backed token representing real-world collateral, ERC-4626 yield vault, Chainlink Proof of Reserve / price feeds, role-gated minting for authorized issuers, DAO governance over asset onboarding and parameter changes, L2 deployment.
— **Option D — On-Chain Prediction Market:** binary outcome markets with an AMM pricing mechanism (LMSR or CPMM), Chainlink oracle for outcome resolution with staleness / dispute window, ERC-1155 outcome share tokens, fee vault for LPs, DAO governance over market creation and resolution disputes, L2 deployment.
— **Option E — Decentralized Insurance Pool:** a risk pool where users stake collateral to underwrite policies, Chainlink oracle feeding the triggering event

(price depeg / weather / liquidation), ERC-4626 vault for underwriter yield, automated claim payouts, DAO governance over accepted risk types and premium parameters, L2 deployment.

2. DELIVERABLES

Submission is a single Git repository (GitHub, public or shared with instructor) containing:

— Smart contract codebase (Foundry or Hardhat project).
— Full automated test suite (unit + fuzz + invariant + fork).
— Frontend dApp (React or HTML/JS with Ethers.js/Viem + Wagmi).
— The Graph subgraph configuration (subgraph.yaml, schema.graphql, mappings).
— Deployment scripts for L2 testnet and verified contract addresses.
— Security audit report (internal, team-authored, minimum 8 pages).
— Architecture & design document with diagrams (minimum 6 pages).
— Gas optimization report with before/after benchmarks.
— README.md.
— Final presentation slide deck (PDF).

3. MANDATORY TECHNICAL REQUIREMENTS

Every item in this section is MANDATORY. Missing any single item is grounds for failure of the entire project regardless of how well the remaining components are implemented. Partial implementations (stubs, TODOs, placeholder functions that are not actually called from tests or the frontend) count as missing.

3.1 Smart Contracts — Required Components

— **Advanced Solidity (Lecture 1):** at least one upgradeable contract using UUPS proxy pattern with a documented V1 → V2 upgrade path; at least one Factory contract using both CREATE and CREATE2; at least one contract with inline Yul assembly that is benchmarked against a pure-Solidity equivalent.
— **Token standards (Lecture 6):** at least one ERC-20 (the governance token MUST be ERC20Votes + ERC20Permit); at least one ERC-721 OR ERC-1155; one ERC-4626 tokenized vault that passes all ERC-4626 rounding invariants.
— **DeFi primitive (Lectures 4 & 5):** a constant-product AMM (x·y=k) with 0.3% fee, slippage protection, and LP tokens, OR a lending pool with LTV, health factor, liquidation, and linear interest rate — one of these must be built from scratch (not forked).

— **Oracles (Lecture 8):** at least one Chainlink price feed integration with staleness check (revert if price older than N seconds); mock aggregator for tests.
— **Indexing (Lecture 8):** a working subgraph indexing your protocol events, with at least 4 entities and 5 documented GraphQL queries.
— **Governance (Lecture 9):** full OpenZeppelin Governor stack — Governor + TimelockController (2-day delay) + ERC20Votes token; voting delay 1 day, voting period 1 week, quorum 4%, proposal threshold 1%; Timelock controls the treasury; full propose→vote→queue→execute lifecycle must be demonstrated end-to-end.
— **Layer 2 (Lecture 7):** all contracts deployed AND verified on Arbitrum Sepolia, Optimism Sepolia, Base Sepolia, or zkSync Sepolia; gas comparison table L1 vs L2 for at least 6 operations.

3.2 Security Requirements

- Every externally callable function must use the Checks-Effects-Interactions pattern OR ReentrancyGuard where applicable — documented in the audit report.
- Every privileged function must use OpenZeppelin AccessControl or Ownable — no unguarded admin functions.
- Slither MUST report zero High and zero Medium findings at submission. All Low and Informational findings must be listed and explicitly justified in the audit report.
- The project must include at least two reproduced-and-fixed vulnerability case studies: one reentrancy and one access-control; before/after tests required.
- No use of tx.origin for authorization. No use of block.timestamp as a source of randomness. No use of deprecated primitives (transfer/send) for ETH — use call{value:} with success check.
- All external calls must handle return values. All ERC-20 interactions must use SafeERC20.

3.3 Testing Requirements

Foundry-based test suite (Hardhat is permitted only if the team justifies it; Foundry is strongly preferred).
Minimum 80 tests total across the repository:

- **Unit tests:** at least 50 — covering every public/external function including revert paths.
- **Fuzz tests:** at least 10 — including the AMM swap function, vault deposit/withdraw, and governance voting power.
- **Invariant tests:** at least 5 — including constant-product invariant (k never decreases on swap), total supply conservation, and treasury accounting.

- **Fork tests:** at least 3 — interacting with real mainnet or testnet protocols (USDC, Uniswap V2 router, Chainlink feed).

Coverage: line coverage ≥ 90% across contracts/ directory, measured with forge coverage. Coverage report must be checked into the repo as a markdown file.

All tests MUST pass in CI on the final commit. A failing test suite at submission = automatic failure.

3.4 Frontend Requirements

— Wallet connection via MetaMask (support for at least WalletConnect as a second connector is a plus).

— Read token balance, voting power, delegate address, and at least one protocol-specific state (pool reserves / vault shares / loan position).

— Write functions: at least 3 state-changing transactions triggered from the UI (e.g., swap, deposit, vote).

— Active proposal list with proposal state (Pending/Active/Succeeded/Defeated/Queued/Executed) and a working vote button.

— Pull indexed data from the subgraph (at least one page/section must read from The Graph, not directly from the contract).

— Proper error handling: transaction rejections, wrong network, insufficient balance must all display a readable user message — no raw RPC errors, no silent failures.

— Network detection: the dApp must detect if the user is on the wrong chain and prompt them to switch.

3.5 DevOps Requirements

— GitHub Actions CI pipeline: on every push and every pull request — install toolchain, compile, run full test suite, run forge coverage, run Slither. PR cannot be merged if CI is red.

— Pre-commit hooks OR a lint step in CI: forge fmt --check, solhint, Prettier for frontend.

— All contracts deployed via a script (deploy.s.sol or deploy.ts). No manual deployment transcripts. The script must be reproducible.

— All contracts verified on the L2 block explorer — links in the README.

4. CODE QUALITY & ENGINEERING STANDARDS

Code that works is not enough. Code must be readable, maintainable, and defensible in a code review. The instructor reserves the right to fail a project on code quality grounds even if the product appears to function.

4.1 DESIGN PATTERNS

The project must demonstrate conscious, documented use of at least FIVE of the following design patterns. Each pattern usage must be justified in the Architecture document:

— Factory pattern (contract deployment)
— Proxy / UUPS (upgradeability)
— Checks-Effects-Interactions
— Pull-over-push payments
— Access Control / Role-based permissions
— Pausable / Circuit Breaker
— State Machine (for protocol lifecycle)
— Oracle adapter / interface abstraction
— Timelock (for governance actions)
— Reentrancy Guard

A pattern dropped in for the sake of a checkbox — with no corresponding need in the protocol — will not count. Over-engineering is penalized equally to under-engineering.

5. COMMIT REQUIREMENTS

Repository must be created and pushed to GitHub by the end of Week 6. Empty repo created at Week 10 = automatic failure.

Commit Message Conventions

Use Conventional Commits (feat:, fix:, test:, docs:, refactor:, chore:, ci:). Messages like "fixed stuff", "asdf", "final final v2" are grounds for deduction.
Example of acceptable commit messages:

— feat(governor): add quorum fraction override to 4%
— fix(amm): correct k-invariant rounding in removeLiquidity edge case
— test(vault): add ERC-4626 inflation-attack fuzz test
— docs(audit): document findings S-03 through S-07

6. DOCUMENTATION REQUIREMENTS

Architecture Document

Minimum 6 pages. Must contain:

— System context diagram (C4 level 1).
— Container / component diagram showing contract relationships, proxy layout, access-control roles, and external dependencies (Chainlink, The Graph, L2).

— Sequence diagrams for at least 3 critical user flows (e.g., swap, propose-vote-execute, deposit-borrow-liquidate).
— Data model: every storage layout for every contract (for upgradeable contracts this is mandatory — storage collisions must be proven impossible).
— Trust assumptions: who can do what, what powers the Timelock has, what powers individual admins have, what happens if the multisig is compromised.
— Design decisions log: for each significant decision, a short ADR (Architecture Decision Record): context, options considered, decision, consequences.

Security Audit Report

Minimum 8 pages, team-authored, structured like a professional audit. Must contain:

— Executive summary.
— Scope (commit hash, files in scope, files out of scope).
— Methodology (tools used, manual review approach).
— Findings table with severity (Critical / High / Medium / Low / Informational / Gas).
— For each finding: title, severity, location (file:line), description, impact, proof of concept, recommendation, status (fixed / acknowledged / wontfix).
— Centralization analysis: who has what powers, what could go wrong if a role-holder is malicious or compromised.
— Governance attack analysis: flash-loan governance attacks, whale attacks, proposal spam, timelock bypass — explain how the design defends against each.
— Oracle attack analysis: price manipulation, stale price, feed depeg.
— Slither output attached as an appendix.

7. DEPLOYMENT & PRESENTATION

— All contracts deployed and verified on an L2 testnet by the submission deadline.
— Deployment script is idempotent and parameterized — the instructor must be able to redeploy to a fresh network using only the script.
— Post-deployment verification script: checks that owner is the Timelock, Timelock delay is correct, Governor parameters match the spec, no admin backdoor remains. Output of this script must be in the repo.

8. MILESTONES

Week	Milestone	Artifact due
End of W6	Team formed, scenario chosen	Team roster
End of W7	Core contracts compile, first tests pass	Repo link, initial CI green
End of W8	DeFi primitive + tokens complete, 50% coverage	Mid-project review checkpoint
End of W9	Governance + oracles + L2 deployment	Testnet addresses + subgraph live
End of W10	Full submission	All deliverables, presentation

Final Presentation

— 15-minute live team presentation + 10-minute Q&A.
— Every team member must present a segment.
— Q&A is individual: the instructor may ask any team member about any part of the codebase. "That was not my part" is not an acceptable answer — every member is expected to understand the whole system at an architectural level.
— Inability to answer questions about code committed under your name = individual deduction up to a failing grade, regardless of the team score.

9. GRADING RUBRIC

Project – 60 points, 40 Points is defence!

Component	Points
Smart contract implementation	20
Security (Slither clean, access control, audit report)	15
Testing (coverage, fuzz, invariant, fork, all passing)	15
Code quality & design patterns	10
Frontend + subgraph integration	10

Deployment & L2 verification	5
Documentation (README + architecture + audit + gas)	10
Git discipline & individual contribution	5
Presentation / Q&A	10